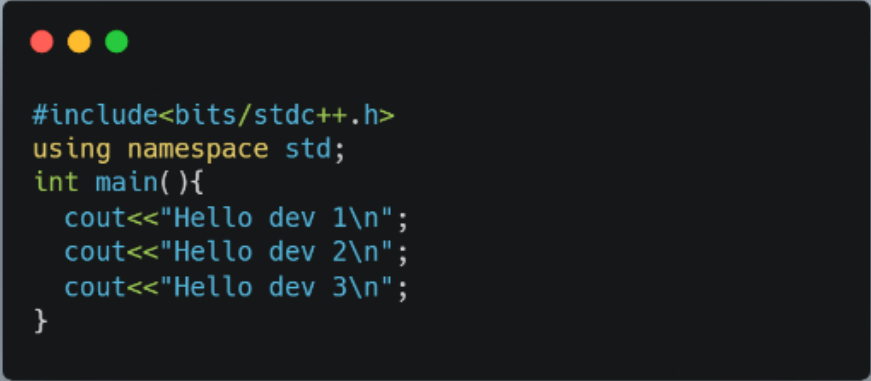


What is time complexity?

Time complexity is a programming term that quantifies the amount of time it takes a sequence of code or an algorithm to process or execute in proportion to the size and cost of input.

It will not look at an algorithm's overall execution time. Rather, it will provide data on the variation (increase or reduction) in execution time when the number of operations in an algorithm increases or decreases.

Yes, as the definition suggests, the amount of time it takes is solely determined by the number of iterations of one-line statements inside the code.



```
#include<bits/stdc++.h>
using namespace std;
int main(){
    cout<<"Hello dev 1\n";
    cout<<"Hello dev 2\n";
    cout<<"Hello dev 3\n";
}
```

Although there are three separate statements inside the **main()** function, the overall time complexity is still **O(1)** because each statement only takes unit time to complete execution.



```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int i,n=35;
    for(i=1;i<=n;i++){
        cout<<"Hello dev 1\n";
        cout<<"Hello dev 2\n";
        cout<<"Hello dev 3\n";
    }
}
```

Here, the **O(1)** chunk of code (the 3 cout statements) is enclosed inside a looping statement which **repeats iteration for 'n' number of times**. Thus our overall time complexity becomes $n \cdot O(1)$, i.e., **O(n)**.

You could say that when an algorithm employs statements that are only executed once, the time required is constant. However, when the statement is in loop condition, the time required grows as the number of times the loop is set to run grows.

When an algorithm has a mix of single-executed statements and LOOP statements, or nested LOOP statements, the time required increases according to the number of times each statement is run.

Understanding various Time Complexity notations

The following are some of the most common asymptotic notations for calculating an algorithm's running time complexity;

- O Notation
- Ω Notation
- θ Notation

Big Oh Notation, O

The systematic way to express the upper limit of an algorithm's running time is to use the Big-O notation $O(n)$. It calculates the worst-case time complexity, or the maximum time an algorithm will take to complete execution.

Also, do remember that this is the **most commonly used notation** for expressing the time complexity of different algorithms unless specified otherwise.

Definition:

Let g and f be functions that belong to the set of natural numbers (N). The function f is said to be $O(g)$, if there is a constant $c > 0$ and a natural number n_0 such that:

$$f(n) \leq cg(n) \text{ for all } n \geq n_0$$

Omega Notation, Ω

The systematic way to express the lower bound of an algorithm's running time is to use the notation $\Omega(n)$. It calculates the best-case time complexity, or the shortest time an algorithm will take to complete.

Definition:

$\Omega(g(n)) = \{ f(n) : \text{there exist positive constants } c \text{ and } n_0 \text{ such that } 0 \leq cg(n) \leq f(n) \text{ for all } n \geq n_0 \}$

The above expression can be defined as a function $f(n)$ which belongs to the set $\Omega(g(n))$ if and only if there exists a positive constant c ($c > 0$) such that it is greater than $c \cdot g(n)$ for a sufficiently large value of n .

The minimum time required by an algorithm to complete its execution is given by $\Omega(g(n))$.

Theta Notation, Θ

The systematic way to express both the lower limit and upper bound of an algorithm's running time is to use the notation $\Theta(n)$.

$\Theta(g(n)) = \{ f(n) : \text{there exist certain positive constants } x_1, x_2 \text{ and } n_0 \text{ such that} \}$

$0 \leq x_1g(n) \leq f(n) \leq x_2g(n) \text{ for all } n \geq n_0 \}$

Let's explore each time complexity type with an example.

1. $O(1)$

Where an algorithm's execution time is not based on the input size n , it is said to have constant time complexity with order $O(1)$.

Whatever be the input size n , the runtime doesn't change. Here's an example:

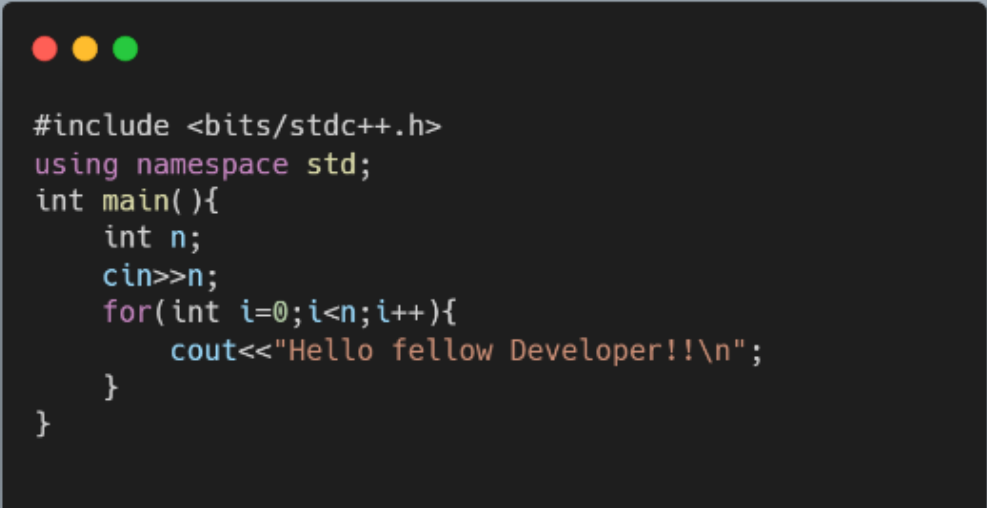
A code editor window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The code is written in C++ and is as follows:

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    cout<<"Hello World!!";
}
```

As you can see, the message "**Hello World!!**" is printed only once. So, regardless of the operating system or computer configuration you are using, the time complexity is constant: $O(1)$, i.e. any time a constant amount of time is required to execute code.

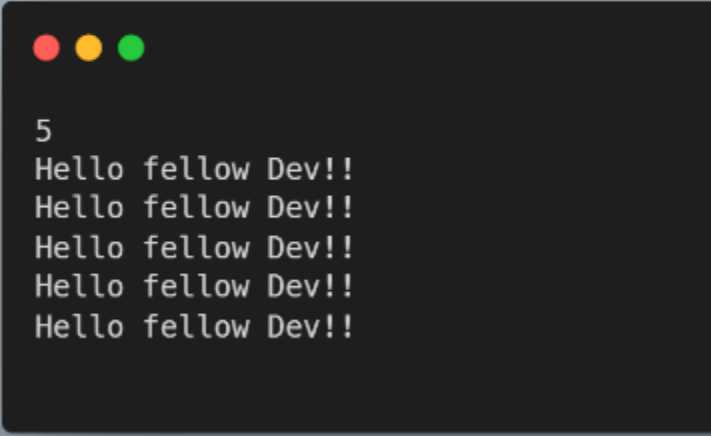
2. $O(n)$

When the running time of an algorithm increases linearly with the length of the input, it is **assumed to have linear time complexity**, i.e. when a function checks all of the values in an input data set (or needs to iterate once through every value in the input), it is said to have a Time complexity of order $O(n)$. Consider the following scenario:

A screenshot of a code editor with a dark background and three colored window control buttons (red, yellow, green) at the top left. The code is written in C++ and demonstrates a loop that iterates from 0 to n-1, printing a message for each iteration.

```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n;
    cin>>n;
    for(int i=0;i<n;i++){
        cout<<"Hello fellow Developer!!\n";
    }
}
```

Output:

A screenshot of a terminal window with a dark background and three colored window control buttons (red, yellow, green) at the top left. The output shows the number 5 followed by five lines of the message "Hello fellow Dev!!".

```
5
Hello fellow Dev!!
Hello fellow Dev!!
Hello fellow Dev!!
Hello fellow Dev!!
Hello fellow Dev!!
```

Based on the above code, you can see that the number of times the statement **"Hello fellow Developer!!"** is displayed on the console is directly dependent on the size of the input variable 'n'.

If one unit of time is used to represent run time, the above program can be run in n times the amount of time. As a result, the program scales linearly with the size of the input, and it has an order of $O(n)$.

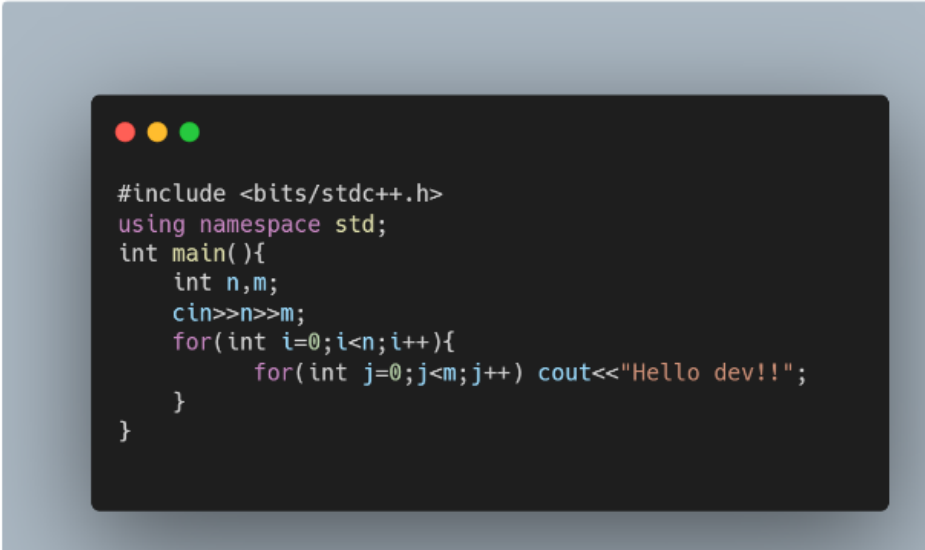
3. $O(n^2)$

When the running time of an algorithm increases non-linearly $O(n^2)$ with the length of the input, it is said to have a non-linear time complexity.

In general, nested loops fall into the $O(n)*O(n) = O(n^2)$ time complexity order, where one loop takes $O(n)$ and if the function includes loops inside loops, it takes $O(n)*O(n) = O(n^2)$.

Similarly, if the function has 'm' loops inside the $O(n)$ loop, the order is given by $O(n*m)$, which is referred to as polynomial time complexity function.

Consider the following program:



```
#include <bits/stdc++.h>
using namespace std;
int main(){
    int n,m;
    cin>>n>>m;
    for(int i=0;i<n;i++){
        for(int j=0;j<m;j++) cout<<"Hello dev!!";
    }
}
```

Output:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top-left corner. The terminal displays the output of a program with two nested loops. The first loop iterates from 3 down to 1, and the second loop iterates from 1 to 2 for each value of the first loop. The output is as follows:

```
3
2
Hello dev!!
Hello dev!!
Hello dev!!
Hello dev!!
Hello dev!!
Hello dev!!
```

As you can see, there are two nested for loops such that the inner loop's complete iteration repeats based on the value of the outer loop. This is the primary reason why you see **"Hello dev!!" printed 6 times (3*2)**.

This amounts to the cumulative time complexity of **$O(m*n)$** or **$O(n^2)$** if you assume that the value of m is equal to the value of n .